

编译器课程答疑对话记录

与 AI 助手的问答记录 (Kimi Code CLI), 主题: ANTLR4 前端使用与 AST 构建。

问: 助教说建议详细阅读 Yx[6] 的代码和指导, 那个在哪?

答: Yx 在 <https://github.com/ZYHowell/Yx> (庄永昊 2020 年的作品), 编译的对象语言是 Mx*——本课程的前身语言。指导手册 compiler-101 里所有“以 Yx 为例”的插图和代码片段都出自这个仓库, 比如 10-semantic.tex 提到的 Yx/src/parser/Yx.g4。

建议按阶段对应着读:

目录/文件	对应阶段	读什么
src/parser/Yx.g4	前端	课程级语言的 g4 长什么样
src/AST/(节点定义)	AST	节点类怎么设计、继承层次怎么分
ASTBuilder 相关	AST	怎么从 ANTLR 树翻成 AST, 最对口
SemanticChecker	语义检查	作用域、类型检查怎么组织
IR、ASM 相关	后续阶段	先不看, 避免信息过载

注意: Yx 的对象语言是 Mx*, 看的是架构和思路, 不是照抄; Code Review 时能讲清“我的设计和 Yx 哪里一样、哪里不一样、为什么”是加分项。

问: visitCrate 中 ctx 的结构是什么, 我怎么遍历 item?

答: CrateContext 的定义在生成代码 build/generated/RustParser.h 里。成员函数是 g4 规则的镜子——规则 crate : innerAttribute* item* EOF ; 右部的每个元素都变成一个访问函数:

g4 元素	生成的访问函数
item*	std::vector<ItemContext*> item() + item(size_t i)
EOF	TerminalNode* EOF()

规律: 出现多次的给 vector + 下标两个版本, 出现一次的给单指针。

遍历方式: for (auto* itemCtx : ctx->item()), 对每个调 visit(itemCtx), 再 any_cast<Item*>。

没覆盖的中间层 (item → visItem → function_) 不会断链: ANTLR BaseVisitor 默认 visitChildren 的 aggregateResult 返回最后一个孩子的结果, 会自动穿透到末端你覆盖的 visitFunction_。

问: 帮我查查我刚写的这个函数有问题。(visitCrate 初版)

答: 两个笔误:

1. std::any_cast<Item*> result; 少了圆括号, 应为 std::any_cast<Item*>(result)——any_cast 是函数模板, 需要调用。这种错误编译器报的是难懂的模板错误, 遇到先检查括号和尖括号。
2. pushback 应为 push_back。

另外提醒: 修好后跑起来 any_cast<Item*> 会抛 std::bad_any_cast, 因为 visitFunction_ 返回的是 Function* 而不是 Item*。std::any 不做继承转换, 存什么类型就必须 cast 什么类型。建议全项目约定: 返回时统一 static_cast 成基类指针 (Item*/Stmt*/Expr*) 再 return。

问：填了，你看看对不对。(visitFunction__)

答：写得对。ctx->identifier()->getText() 取函数名, any_cast<Block*>(visit(ctx->blockExpression())) 取函数体、结尾 static_cast<Item*> 遵守约定，三处都正确。编译运行后输出 Crate → Function main，链路打通。没有 Block 内容是因为 visitBlockExpression 还是空壳。

问：注释里说解析树里 block 的语句藏在 statements 规则下面，可是我没找到。

答：链路是 blockExpression → statements → statement+。g4 规则：

```
blockExpression : LCURLYBRACE innerAttribute* statements? RCURLYBRACE ;
statements      : statement+ expression? | expression ;
```

对应的 Context 访问函数：ctx->statements() (可空！空 block 返回 nullptr) → ->statement() (语句 vector)。statements 是包装规则，让 block、if 体、while 体复用“一组语句 + 可选尾表达式”的结构。

两个提示：statements? 带问号，遍历时判空；statement+ expression? 里落单的 expression? 是尾表达式 (fn f() -> i32 { 1 + 2 } 的 1+2)，之后处理。

问：写完了那几个函数，帮我 debug。(跑 tests/minimal.rs 崩溃：bad_any_cast)

答：两类 bug：

Bug 1 (崩溃直接原因)：三个函数返回时忘了转基类指针。visitLetStatement 存的是 LetStmt* 而调用方 cast Stmt*；visitLiteralExpression 存 Literal*；visitArithmeticOrLogicalExpression 存 BinaryExpr*。修法：return 前 static_cast<Stmt*>(node) / static_cast<Expr*>(node)。

Bug 2 (修完后会撞的空指针)：ctx->statements() (空 block)、ctx->type_() (省略类型标注)、ctx->expression() (无初值 let) 三处可空未判空。

问：改好了，现在对了么？

答：基类转换改对了，但两个空判断写错了位置——判空要判指针本身：

```
if (ctx->statements()->statement()) // statement() 返回 vector, 不是 bool
if (ctx->statements())              // 判 statements 这个指针

if (ctx->type_()->getText())         // getText() 返回 string; 且空指针调方法本身就崩
if (ctx->type_())                   //
```

规律：g4 里带 ? 的元素生成“可能为空的指针”，判空永远是 if (ctx->xxx())，在 if 里面才调它的方法。

问：好了，重新编译跑一下看看。

答：四个测试全部通过：minimal.rs 输出完整 6 节点树；let y = 5; (无类型标注) 正确；fn main() {} (空 block) 不崩；let z: i32; (无初值) 正确。第一个里程碑完成。

问：ast 文件夹里还有几个 TODO，下面就是要做那一部分吗？

答：按时间分类：

- 现在做：ast.h 的扩展节点提示 (PathExpr→IfExpr→WhileExpr→ReturnStmt→CallExpr→StructDef)；Function 的参数和返回类型；Block 尾表达式 (fn f() -> i32 { 1+2 } 这种写法测试一定会出现)
- 语义检查期：给节点挂类型信息/源码位置
- IR 阶段：Literal 枚举分类 (int/bool/char/string)
- 另有 ast.h:62 注释过时 (省略类型标注已支持)，语义应为“类型由语义检查阶段推导填充”

问：W4 检查内容更改，语义检查放到下一次，当务之急是建完 AST 树，详细讲讲 AST 扩展。

答：先配调试工具：给 main 加 --tree 开关打印 ANTLR 解析树。然后按波次扩展：

- 第 1 波（表达式补全）：PathExpr（变量引用，链路 pathExpression → pathInExpression → pathExprSegment → pathIdentSegment → identifier 四层穿透）；比较/赋值复用 BinaryExpr；一元运算新建 UnaryExpr；ExprStmt
- 第 2 波（控制流）：IfExpr（注意 Rust 里 if 是表达式；else if 链的表示决策）；WhileExpr/LoopExpr（loopExpression 是四选一的壳，for 先查 spec 要不要）；Return/Break/Continue
- 第 3 波（函数完整化）：参数（functionParameters → functionParam → functionParamPattern）、返回类型、CallExpr（左递归后缀）、Block 尾表达式
- 第 4 波（复合类型）：StructDef、结构体字面量、FieldExpr、ImplBlock、MethodCallExpr
- 第 5 波：数组、引用、const/static；enum/match 视 spec 而定

三条纪律：每波结束跑全部旧测试；每波提交一次 git；解析树为准，别猜。

问：帮我把 main 的开关加好，另外这些扩展应该在哪个文件里实现？给一点代码骨架。

答：每种新语法固定动 4 个文件：ast.h/ast.cpp（节点类 + accept）、ast_visitor.h（visit 声明）、ast_printer.cpp（打印）、ast_builder.cpp（visit 实现）。已加好第 1、2 波骨架（8 个新节点、11 个 visit 空壳带伪代码）和 --tree 开关（./build/compiler x.rs --tree 看解析树）。

问：怎么区别 minus 和 not？（visitNegationExpression 里）

答：g4 规则（MINUS | NOT）expression 里的每个备选 token 都会生成同名访问函数，出现则非空：

```
if (ctx->MINUS()) node->op = "-";
if (ctx->NOT())   node->op = "!";
```

通用规律：判断“这条规则实际匹配了哪个分支”，就是看哪个访问函数返回非空。另外 -、== 在词法层就是不同的 token（MINUS、MINUSEQ），不用自己拆字符串。

问：怎么使得 if/while/loop 走的是 expressionWithBlock 分支？

答：路由是 grammar 自动完成的。语句的解析路径：

statement → expressionStatement → expressionWithBlock → ifExpression

expressionStatement：expression SEMI | expressionWithBlock SEMI？——普通表达式必须带分号；if/while/loop 这类“自带大括号”的结构分号可有可无。expressionWithBlock 是纯包装规则，归一了 blockExpression、ifExpression、loopExpression、matchExpression。

代码里不用区分分支：判空后 visit 任一非空孩子即可，中间层靠默认为自动穿透到末端的 visitIfExpression。

另外注意姊妹入口：expression 规则里也有 expressionWithBlock # ExpressionWithBlock_，所以 let x = if c {1} else {2}；这种 if 出现在表达式位置的写法也能自动穿透。

问：如何判断 IfExpression 里的 else 后面是否有嵌套？

答：IfExpressionContext 的关键成员：blockExpression() 是 vector（blockExpression(0) 是 then，blockExpression(1) 是 else 块）；ifExpression() 非空 = else if 链；KW_ELSE() 非空 = 有 else。

判断顺序有讲究——先查 ifExpression() 再数 blockExpression().size()：else if 情况下本层 blockExpression 只有 1 个（嵌套 if 的块挂在嵌套 ctx 上），和“无 else”无法区分。

问: `ast_builder.cpp` 里几个新的函数都写完了, 帮我看看对不对。

答: 六类问题:

1. `visit()` 忘了 `any_cast` (多处): `visit` 永远返回 `std::any`, 进字段前必须拆包
2. 把解析树节点直接赋给 `AST` 字段 (`node->elseBlock = ctx->blockExpression(1)`): 两棵树两个世界, 唯一转换通道是 `visit + any_cast`
3. 字段名丢了: `node = ...` 应为 `node->expr = ...`
4. 访问函数形态记错: 只出现一次的元素是单指针, 无下标版本
5. `else if` 分支逻辑错: 嵌套 `if visit` 一次即可 (递归自动建好子树), 不用遍历内部; 且 `visit` 返回 `Expr*`, `any_cast<ExprStmt*>` 运行期必炸——应该 `new ExprStmt` 再把 `Expr*` 塞进它的字段
6. 复制粘贴错误: `visitBreakExpression` 里 `new ReturnExpr()` 应为 `BreakExpr`——编译器查不出, 测试时 `break` 会打印成 `ReturnExpr`, 复制后逐字检查标识符

问: 现在再看看还有没有问题?

答: 剩 5 个错误同一类: `any_cast` 类型和目标 `visit` 实际返回的类型不匹配。判断方法: 看目标函数的 `return` 语句——`visitBlockExpression` 返回 `Block*`, 所以 `block` 相关 `cast` 成 `Block*`; 表达式 `visit` 都返回 `Expr*`。项目里流动的指针类型有五种: `Crate*`、`Item*`、`Stmt*`、`Expr*`、`Block*` (`Block` 独立, 既不是 `Stmt` 也不是 `Expr`)。写 `any_cast` 时先跳到目标 `visit` 的 `return` 看一眼。

问: 现在还有问题吗?

答: 编译通过。综合测试 (`if/else if/else` 嵌套、`while`、`loop`、`return/break/continue`、一元负号、比较、赋值、变量引用) 全部正确。但 `tests/hello.rs` 崩溃——引出宏调用问题。

问: (`hello.rs` 崩溃: `bad_any_cast`)

答: `println!("{}", y)`; 走 `statement` 的第 5 个备选 `macroInvocationSemi`, 未覆盖时产生连锁穿透: `visit` → 默认穿透 → 一直钻到叶子 `token` → `visitTerminal` 返回空 `any` → 空 `any` 冒泡 → `any_cast` 抛 `bad_any_cast`。

重要调试结论: `bad_any_cast` 第一嫌疑是“有语法结构没覆盖”, 第二才是类型不匹配。

修法: `println!` 是课程测试程序的输出手段, 必须支持。加 `MacroStmt` 节点 (`name + argsText` 原文), 覆盖 `visitMacroInvocationSemi`; 参数是 `tokenTree` (原始 `token` 流不是表达式), 先存原文, `CodeGen` 阶段再解析。

问: 前两步我做完了, 帮我检查, 完成第 3 步机械步骤, 加固也一起做了。

答: 你的实现有三处问题已修正: 挂错了 `visit`——`println!()`; 走 `macroInvocationSemi` (语句级), 不是 `MacroInvocationAsExpression`; `ctx->args()` 不存在, 改为遍历 `tokenTree()` 拼接; `MacroStmt` 是 `Stmt`, 返回处 `static_cast` 成 `Stmt*`。另外补了头文件声明。

机械三件套 (`visitor` 声明、`accept`、`printer`) 已配好。防御加固: `visitBlockExpression` 和 `visitCrate` 都加了 `result.has_value()` 检查, 未覆盖的结构打印 `[ast]` 未处理的... 警告并跳过。

验证: `minimal/wave12/hello` 全过; `syntax-tour.rs` 正确报告 5 个未处理顶层条目 (`const`、`static`、`struct`、`impl`、`enum`)。表达式内部的崩溃 (元组、数组、`match`) 是“下一个任务指示器”, 属正常。

问: 现在首要任务是完成 `AST` 树建造, 下一步做什么, 离建成还有多远?

答: 进度约一半节点量, 但所有机制套路已打通, 剩下是堆量。

波次	内容
第 3 波	函数参数 + 返回类型、 <code>CallExpr</code> 、 <code>Block</code> 尾表达式 (约 1 天)

波次	内容
第 4 波	StructDef、结构体字面量、字段访问、ImplBlock、方法调用 (1-2 天)
第 5 波	数组、引用、const/static (约 1 天)
视 spec	enum/match、for、元组——先查 RCompiler-Spec 定范围

下一步第 3 波：真实测试程序里全是函数调用，且 Function 加参数是 struct/impl 的前置。另外该抽空翻 spec 确认子集范围，防止多做或漏做。

问：可以加第 3 波的骨架了。

答：已加好。ast.h 新增 Param 结构、Function 的 params/returnType 字段、Block 的 tailExpr、CallExpr 节点；打印器升级为显示完整函数签名。AstBuilder 留四个 TODO (带伪代码)：参数列表 (functionParameters→functionParam→functionParamPattern 链路，pattern 取名和 letStatement 同理)、返回类型 (functionReturnType 判空)、尾表达式 (statements()->expression() 判空)、visitCallExpression (callee 是 ctx->expression() 左递归，实参在 callParams()->expression() vector，callParams 本身可空)。

问：ast_builder.cpp 里的 TODO 都做完了，帮我检查。

答：两个问题：

1. 编译错误：functionParam() 返回 vector 不是单指针，需要循环；而且只 new 了一个 Param 没 push 进 params。Param 是纯数据 struct，按值 push_back 即可。
2. 隐蔽 bug：functionReturnType : RARROW type_——getText() 会把箭头也拿进去 (->i32)，应取 ctx->functionReturnType()->type_()->getText()。教训：getText() 返回整棵树原文，包括你不想要的符号。

提醒：第 4 波做 impl 时 functionParameters 里会混进 selfParam，和普通参数分开处理。

问：改完了，帮我验证，然后搭第四波骨架。

答：第 3 波全部验证通过：参数、返回类型 (无箭头)、尾表达式、CallExpr (有/无实参)、全部回归。第 4 波骨架已搭：StructDef (字段复用 Param)、ImplBlock (装 Function* 方法列表)、StructLiteralExpr ((字段名, 初值) 有序对)、FieldExpr (p.x)、MethodCallExpr (receiver+ 方法名 +args, 不含 self)。5 个 visit 空壳带伪代码。关键难点提示：selfParam 不出现在 functionParam 列表里，方法调用时 receiver 是隐式 self 实参，影响 CodeGen。

问：写好了，帮我 debug 一下，另外我对第 4 波的这些函数起什么作用不是很清楚，可能是对 Rust 的语法理解问题？

答：debug 结果——visitInherentImpl、visitFieldExpression、visitMethodCallExpression 全对；两处错误：

1. visitStructStruct : new Param() 应为值类型 (fields 是 vector<Param>)；field->identifier() 是解析树节点指针，要 ->getText()
2. visitStructExpression_ : push_back 的 pair 里两个都是解析树指针，要分别 getText() 和 any_cast+visit

概念讲解：C++ 里数据和函数写在 class 里；Rust 分开写——struct 只装数据，impl 块“给类型挂上函数”，因为 Rust 没有继承。五个 visit 的分工：visitStructStruct 成员变量声明、visitInherentImpl 成员函数声明 (写在类外面)、visitStructExpression_ 聚合初始化、字段访问和方法调用与 C++ 相同。self 的理解关键：p.shift(3) 表面一个实参，p 被隐式传入 (C++ 的 this)，这影响 CodeGen 的参数槽位。

问：field 和 method 在 Rust 里是什么意思？(改好了，看一下还有问题不)

答：验证结果：struct/impl/字面量/字段访问/方法调用全部正确。但发现两个隐藏问题：

1. 严重（静默出错）：`self.x += dx` 被吞成 `PathExpr dx`——复合赋值走 `CompoundAssignmentExpression` 标签未覆盖，默认穿透“返回最后一个孩子”导致整个表达式被静默替换。教训：防御 `guard` 只能拦空 `any`，拦不住错误穿透；凡一个规则有多个有意义的孩子，必须显式覆盖。
2. 多段路径只显示第一段（`Point::new → Point`）。

概念回答：field= 成员变量（struct 里的数据，`p.x` 访问，和 C++ 一样）；method= 第一个参数是 `self` 的函数（用 `.` 调用，`p` 隐式传入）。对照：关联函数（无 `self`，如 `Point::new`） C++ 静态成员函数，用 `::` 调用。

问：你帮我把现有问题改好，并用注释标注改动痕迹，然后搭最后一波的框架。

答：两处修复（注释已标注）：新增 `visitCompoundAssignmentExpression`（`op` 用 `compoundAssignOperator` 子规则一步拿到）；`visitPathExpression` 改为遍历 `pathExprSegment()` `vector` 用 `::` 拼接，`Point::new` 完整显示。

第 5 波骨架：`ArrayExpr`（列举 `[1,2,3]` 与重复 `[0;5]` 两形式，用 `arrayElements()->SEMI()` 是否非空区分）、`IndexExpr`、`ConstDef`、`StaticDef`（多 `isMut` 字段）；引用 `&x/&mut x/*p` 复用 `UnaryExpr`。6 个 `visit` 空壳带伪代码。全部回归通过。

问：**GitHub** 现在似乎可以了，尝试帮我交一次。

答：推送仍失败，但定位到根因：不是 **GitHub** 的问题，是代理软件把 `github.com` 路由到了失效节点（代理本身活着，百度正常；`github.com/api/codeload` 全断）。解决方案：在代理客户端切换 **GitHub** 规则到可用节点；或从 Windows 侧用 `PowerShell` 直接推（仓库在 `D:\Compiler`）。本地 9 个提交都在，不会丢。

（安全提醒：对话中出现的 **GitHub** token 已多次明文暴露，使用完毕后应立即在 **GitHub** 设置中吊销。）